Lab Report for Lab 4

Yat Long Chan

Source code link: https://github.com/ychan05/cs391r1-lab4

Lab 3 Problem 2.3

1. Approach

```
module control_unit #(
    parameter WIDTH = 32,
    parameter NUM_REGS = 16
)(
    input wire clk,
input wire rst,
output wire error,

// BRAM Manager
output wire [19:0] M_AXI_AWADDR,
output wire [2:0] M_AXI_AWPROT,
output wire M_AXI_AWVALID,
input  wire M_AXI_AWREADY,

output wire [31:0] M_AXI_WDATA,
output wire [3:0] M_AXI_WSTRB,
output wire M_AXI_WVALID,
input  wire M_AXI_WREADY,

input  wire [1:0] M_AXI_BRESP,
input  wire M_AXI_BVALID,
output wire M_AXI_BREADY,
```

```
output wire [19:0] M_AXI_ARADDR,
output wire [2:0] M_AXI_ARPROT,
output wire M_AXI_ARVALID,
input  wire M_AXI_ARREADY,

input  wire [31:0] M_AXI_RDATA,
input  wire [1:0] M_AXI_RRESP,
input  wire M_AXI_RVALID,
output wire M_AXI_RREADY
);
```

```
// Program Counter
reg [19:0] PC;

// BRAM control signals
reg arvalid_reg;
reg rready_reg;

// Instruction register
reg [31:0] fetched_instruction;

// Assign BRAM outputs
assign M_AXI_ARADDR  = PC; // always read from address stored in PC
assign M_AXI_ARPROT  = 3'b000;
assign M_AXI_ARVALID = arvalid_reg; // take vals from reg since they need to be changed in FSM
assign M_AXI_RREADY  = rready_reg; // takes vals from reg since they need to be changed in FSM

// Zero out unused write channels
assign M_AXI_AWADDR  = 20'b0;
assign M_AXI_AWPROT  = 3'b0;
assign M_AXI_AWVALID = 1'b0;
assign M_AXI_WDATA   = 32'b0;
assign M_AXI_WSTRB   = 4'b0;
```

```
assign M_AXI_WVALID  = 1'b0;
assign M_AXI_BREADY  = 1'b0;
```

```
always @(posedge clk) begin
    if (rst) begin
        state <= 0;
        error_reg <= 0;
        we <= 0;
        arvalid_reg <= 0;
        rready_reg <= 1;
        PC <= 0;
        fetched_instruction <= 0;
    end else begin
        case (state)
            3'd0: begin // STATE 0: READY
                we <= 0;
                error_reg <= 0;
                arvalid_reg <= 1; // Ready to give address to read from
                rready_reg <= 1;  // Set ready to read data
                state <= 1;
            end

            3'd1: begin // STATE 1: FETCHING
                we <= 0;
                error_reg <= 0;

                if (M_AXI_RVALID && M_AXI_RREADY) begin
                    arvalid_reg <= 0; // stop giving instruction to AXI
                    fetched_instruction <= M_AXI_RDATA;
                    state <= 2;
                end
            end

            3'd2: begin // STATE 2: DECODE
                // Decode instruction
```

```
            opcode <= fetched_instruction[6:0];
            funct7 <= fetched_instruction[31:25];
            funct3 <= fetched_instruction[14:12];
            rs1 <= fetched_instruction[19:15];
            rs2 <= fetched_instruction[24:20];
            rd <= fetched_instruction[11:7];

            state <= 3;
        end
        ...
        3'd4: begin // STATE 4: WRITEBACK
            if (error_reg) begin
                we <= 0;
                arvalid_reg <= 0;
                rready_reg <= 0;
                PC <= PC;
            end else begin
                // Select data to write based on opcode
                if (opcode == 7'b0110111) begin // LUI
                    d_in <= imm;
                    we <= 1;
                end else begin // R-type and I-type
                    d_in <= alu_res;
                    we <= 1;
                end

                state <= 5;
            end
        end

        3'd5: begin // STATE 5: MOVE TO NEXT
            we <= 0;
            PC <= PC + 4;
            state <= 0;
        end
```

```
            default: state <= 0;
        endcase
    end
end
```

I started by adding the signals needed for the AXI BRAM manager which will be the inverse of the outputs and inputs of the subordinate. I then created a program counter, registers to control the AXI BRAM outputs, and assigned the registers to the corresponding wires. I also zeroed out the write signals. In my FSM, I added 2 new states. One is used to fetch the instruction from the AXI BRAM and the second is after writeback, to increment the PC by 4. In my writeback stage, I also added a conditional statement to check if error is high. If it is, then the program halts. I fixed the assignments to we signal to be non blocking to ensure that the data being written to the register file was correct.

2. Testing

```
    // Instantiate the control unit
    control_unit dut (
        .clk(clk),
        .rst(rst),
        .error(error),

        .M_AXI_AWADDR(awaddr),
        .M_AXI_AWPROT(awprot),
        .M_AXI_AWVALID(awvalid),
        .M_AXI_AWREADY(awready),

        .M_AXI_WDATA(wdata),
        .M_AXI_WSTRB(wstrb),
        .M_AXI_WVALID(wvalid),
        .M_AXI_WREADY(wready),

        .M_AXI_BRESP(bresp),
        .M_AXI_BVALID(bvalid),
        .M_AXI_BREADY(bready),
```

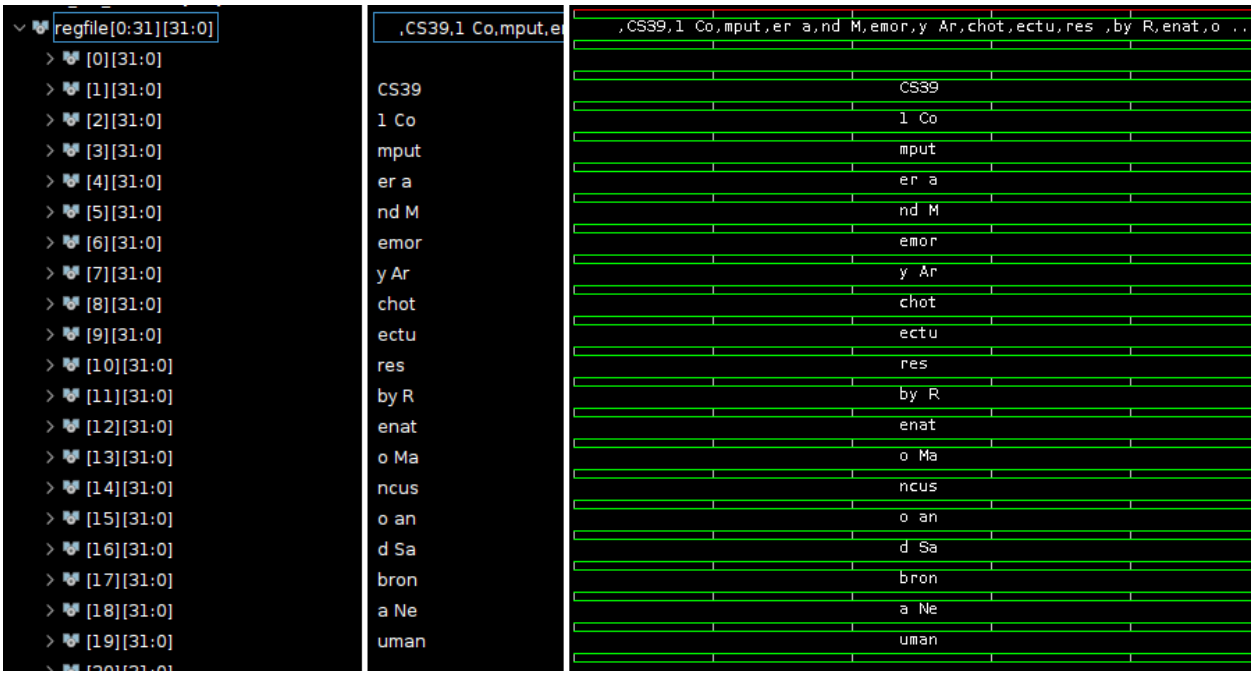
```
    .M_AXI_ARADDR(araddr),
    .M_AXI_ARPROT(arprot),
    .M_AXI_ARVALID(arvalid),
    .M_AXI_ARREADY(arready),

    .M_AXI_RDATA(rdata),
    .M_AXI_RRESP(rresp),
    .M_AXI_RVALID(rvalid),
    .M_AXI_RREADY(rready)
);


// Instantiate a simple BRAM model
bram_controller bram (
    .s_axi_aclk(clk),
    .s_axi_aresetn(~rst),
    .s_axi_araddr(araddr),
    .s_axi_arprot(3'b000),
    .s_axi_arvalid(arvalid),
    .s_axi_arready(arready),
    .s_axi_awaddr(awaddr),
    .s_axi_awprot(3'b000),
    .s_axi_awvalid(awvalid),
    .s_axi_awready(awready),
    .s_axi_wdata(wdata),
    .s_axi_wstrb(4'b1111),
    .s_axi_wvalid(wvalid),
    .s_axi_wready(wready),
    .s_axi_bresp(bresp),
    .s_axi_bvalid(bvalid),
    .s_axi_bready(bready),
    .s_axi_rdata(rdata),
    .s_axi_rresp(),
    .s_axi_rvalid(rvalid),
```

```
        .s_axi_rready(rready)
    );
```

```
        // Load program into memory
        $readmemh("/home/ugrad/yc3146/lab3/lab3.srcs/sim_1/imports/new/lab
3_binary.hex", my_memory);

        // Load program into BRAM over AXI (byte to 32-bit word)
        for (int i = 0; i < 512; i = i + 4) begin
            awvalid = 1;
            wvalid  = 1;
            awaddr  = i;
            wdata   = {my_memory[i+3], my_memory[i+2], my_memory[i+1], my_m
emory[i]};
            #20;
            awvalid = 0;
            wvalid  = 0;
            bready  = 1;
            #20;
            bready  = 0;
            #30;
        end
```

I used the reference code for loading the hex file into the BRAM. I instantiated the control unit with the same signals that the the AXI BRAM instantiation also uses. This makes it so that the control unit and the BRAM loading occurs concurrently. Because of this, I had the test bench stall for 30 time units a the end to ensure that the correct data was being written while the control unit was trying to access a specific address.

3. Results



The waveforms indicate that the control unit is functioning correctly with the AXI BRAM module. Instructions are fed through correctly and operations are performed. Having an error halts the program as expected. In the control unit, the read address stays at the 00160 address while other addresses are being written to in BRAM. If the Control unit was not halted, the control unit would be trying to read from the same addresses being written to.

4. Reflections

Something I found difficult for this section was understanding that the inputs and outputs were reversed for the control unit and BRAM ports. Since I connected the BRAM and control unit in the test bench, I had to take extra considerations to make sure that the instructions were loaded and the control unit was taking them in at the same and correct times.

Lab 4 Problem 2.1

1. Approach

```
reg [31:0] loaded_data; // store data from load instructions
// FSM: 0 = READY, 1 = FETCHING, 2 = DECODE, 3 = EXECUTE, 4 = WRITEBA
CK, 5 = MOVE TO NEXT,
// 6 = LOAD, 7 = load fetch,

reg [3:0] state;
...
          4'd3: begin // STATE 3: EXECUTE
             case (opcode)
                  ...
               7'b0000011: begin // I type loads
                  // first calculate address to read from
                  alu_op1 <= rs;
                  alu_op2 <= {{20{fetched_instruction[31]}}, fetched_instruction
[31:20]};
                  alu_control <= 4'b0111;
                  state <= 6; // move to next state to load because we need one c
lock cycle to calc address
                    ...
               end
               ...
                4'd6: begin // STATE 6: LOAD
                        araddr_reg  <= alu_res[19:0]; // take the calculated address
                        arvalid_reg <= 1;
                        rready_reg <= 1;
                        state <= 7;

                    end

          4'd7: begin // STATE 7: Load fetch
           if (M_AXI_RVALID && M_AXI_RREADY) begin
              arvalid_reg <= 0;
              loaded_data <= M_AXI_RDATA;
              state <= 4; //  move to WRITEBACK
```

```
          end
        end
        ...
         4'd4: begin // STATE 4: WRITEBACK
           if (error_reg) begin
              we <= 0;
              arvalid_reg <= 0;
              rready_reg <= 0;
              PC <= PC;
           end else begin
              // Select data to write based on opcode
              if (opcode == 7'b0110111) begin // LUI
                 d_in <= imm;
                 we <= 1;
              end
              else if (opcode == 7'b0000011) begin

                 case(funct3)
                    3'h0: d_in <= {{24{loaded_data[7]}},  loaded_data[7:0]};   // L
B
                    3'h1: d_in <= {{16{loaded_data[15]}}, loaded_data[15:0]};  // L
H
                    3'h2: d_in <= loaded_data; // LW
                    default: d_in <= 0;
              endcase
              we <= 1;
              end
              else begin // R-type and I-type
                 d_in <= alu_res;
                 we <= 1;
              end

              state <= 5;
           end
        end
```

We have to wait for the ALU to finish calculating the address and also the AXI BRAM handshake to complete. I create 2 more states to handle this. After the load address is calculated by the ALU, we use thatt address to read from the

2. Testing

```
module control_unit_tb();
    parameter WIDTH = 32;
    parameter NUM_REGS = 32;
    bit clk;
    bit rst;
    wire awready;
    bit awvalid;
    bit[19:0] awaddr;
    wire wready;
    bit wvalid;
    bit[31:0] wdata;
    bit bready;
    wire bvalid;
    wire[1:0] bresp;
    wire arready;
    bit arvalid;
    bit[19:0] araddr;
    bit rready;
    wire rvalid;
    wire[31:0] rdata;

    reg _rst;
    reg _awvalid;
    reg[19:0] _awaddr;
    reg _wvalid;
    reg[31:0] _wdata;
    reg _bready;
    reg _arvalid;
    reg[19:0] _araddr;
    reg _rready;
```

```
always #5ns begin
    clk = ~clk;
end
always @ (posedge clk) begin
    _rst <= rst;
    _awvalid <= awvalid;
    _awaddr <= awaddr;
    _wvalid <= wvalid;
    _wdata <= wdata;
    _bready <= bready;
    _arvalid <= arvalid;
    _araddr <= araddr;
    _rready <= rready;
end
bram_controller my_bram(
    .s_axi_aclk(clk),
    .s_axi_aresetn(~rst),
    .s_axi_araddr(_araddr),
    .s_axi_arprot(0),
    .s_axi_arready(arready),
    .s_axi_arvalid(_arvalid),
    .s_axi_awaddr(_awaddr),
    .s_axi_awprot(0),
    .s_axi_awready(awready),
    .s_axi_awvalid(_awvalid),
    .s_axi_bready(_bready),
    .s_axi_bresp(bresp),
    .s_axi_bvalid(bvalid),
    .s_axi_rdata(rdata),
    .s_axi_rready(_rready),
    .s_axi_rvalid(rvalid),
    .s_axi_wdata(_wdata),
    .s_axi_wready(wready),
    .s_axi_wstrb('b1111),
    .s_axi_wvalid(_wvalid)
```

```
);


reg control_unit_error;
reg [2:0] M_AXI_AWPROT;
control_unit #(.WIDTH(WIDTH), .NUM_REGS(NUM_REGS)) dut (
        .clk(clk),
        .rst(rst),
        .error(control_unit_error),

        .M_AXI_AWADDR(awaddr),
        .M_AXI_AWPROT(awprot),
        .M_AXI_AWVALID(awvalid),
        .M_AXI_AWREADY(awready),

        .M_AXI_WDATA(wdata),
        .M_AXI_WSTRB(wstrb),
        .M_AXI_WVALID(wvalid),
        .M_AXI_WREADY(wready),

        .M_AXI_BRESP(bresp),
        .M_AXI_BVALID(bvalid),
        .M_AXI_BREADY(bready),

        .M_AXI_ARADDR(araddr),
        .M_AXI_ARPROT(arprot),
        .M_AXI_ARVALID(arvalid),
        .M_AXI_ARREADY(arready),

        .M_AXI_RDATA(rdata),
        .M_AXI_RRESP(rresp),
        .M_AXI_RVALID(rvalid),
        .M_AXI_RREADY(rready)
    );
        wire [WIDTH-1:0] regfile [0:NUM_REGS-1];
        genvar i;
```

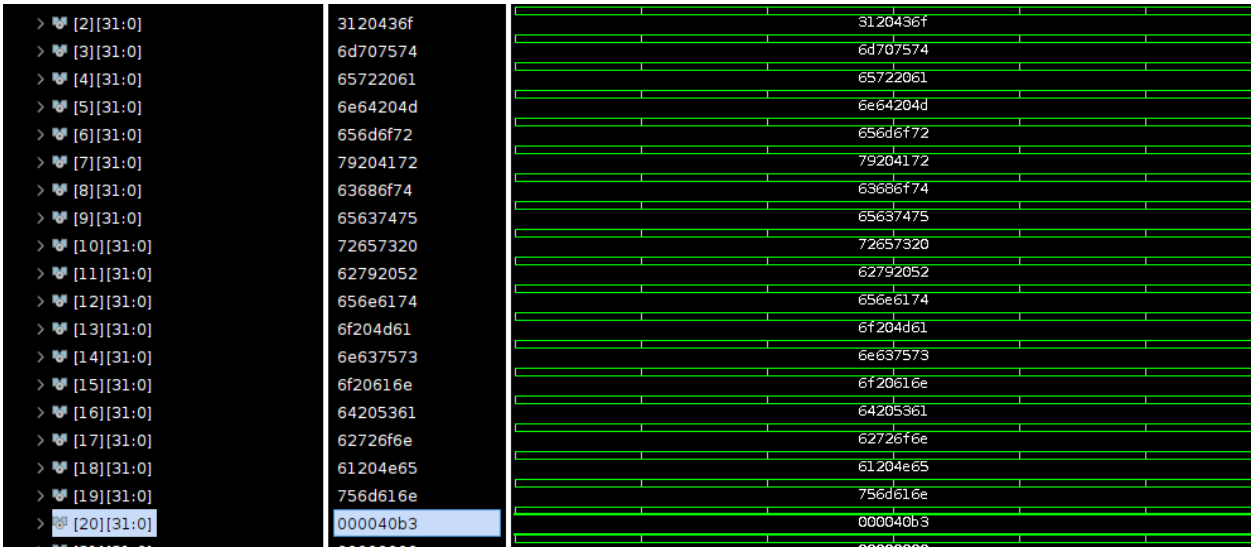
```
        generate
            for (i = 0; i < NUM_REGS; i = i + 1) begin : expose_regs
                assign regfile[i] = dut.regfile.the_regs[i];
            end
        endgenerate
    reg [7:0] my_memory[511:0];
    initial begin
        rst = 1;
        #20ns;
        rst = 0;
        #20ns;
        $readmemh("/home/ugrad/yc3146/lab3/lab3.srcs/sources_1/new/lab3_bi
nary.hex", my_memory);
        #40ns;
        for (int i = 0; i < 512; i+=4) begin
        awvalid = 1;
        wvalid = 1;
        awaddr = i;
        wdata = {my_memory[i+3], my_memory[i+2], my_memory[i+1], my_memo
ry[i]};
        #20ns;
        awvalid = 0;
        wvalid = 0;
        bready = 1;
        #20ns;
        bready = 0;
        #20ns;
    end

    #20ns;
    // actual test-bench starts here...
    rst = 1;
    #10;
    rst = 0;

    #1000000;
```

```
    $finish;

end
endmodule
```

```
lab3_binary.hex:
...
03 2A 00 00
```

I changed the test bench to load instructions first before running the program on the control unit. I also addded a load word instruction to load the instruction at memory address 0 to register 20

3. Results



This waveform shows that the correct data is loaded into register 20 from memory address 0.

4. Reflections

Something that was difficult for this portion was making a test for the load instructions. At first I tried to directly write a separate piece of data to the memory and load that using an instruction but I was unable to figure it out and decided to just load something that I knew was guaranteed to be stored in memory correctly.

Lab 4 Problem 2.2

1. Approach

```
// assign write channel registers
assign M_AXI_AWADDR  = awaddr_reg;
assign M_AXI_AWPROT  = 3'b000;
assign M_AXI_AWVALID = awvalid_reg;
assign M_AXI_WDATA   = wdata_reg;
assign M_AXI_WSTRB   = wstrb_reg;
assign M_AXI_WVALID  = wvalid_reg;
assign M_AXI_BREADY  = bready_reg;
...
        4'd3: begin // STATE 3: EXECUTE
            case (opcode)
              ...
               7'b0100011: begin // S type
                     alu_op1 <= rs;
                     alu_op2 <= {{20{fetched_instruction[31]}}, fetched_instructio
n[31:25], fetched_instruction[11:7]};
                     alu_control <= 4'b0111;
                     state <= 8; // store state after calculating address
               end
               ...
        4'd8: begin // STORE: issue address and data to AXI
            awaddr_reg  <= alu_res[19:0];
            awvalid_reg <= 1;
            wvalid_reg  <= 1;

            case (funct3)
               3'h0: begin // SB
                  wdata_reg <= {24'b0, rt[7:0]};
                  wstrb_reg <= 4'b0001;
               end
               3'h1: begin // SH
```

```
                wdata_reg <= {16'b0, rt[15:0]};
                wstrb_reg <= 4'b0011;
            end
            3'h2: begin // SW
                wdata_reg <= rt;
                wstrb_reg <= 4'b1111;
            end
            default: error_reg <= 1;
        endcase

    if (M_AXI_AWREADY && M_AXI_WREADY) begin
            awvalid_reg <= 0;
            wvalid_reg  <= 0;
            state <= 9; // move to wait for BVALID
        end
    end

    4'd9: begin
    if (M_AXI_BVALID) begin
                bready_reg <= 1;
                awvalid_reg <= 0;
                wvalid_reg <= 0;
                state <= 10;
            end

    end
    4'd10: begin // STORE wait for B
            bready_reg <= 0;
            state <= 5; // increment PC
    end
```

I first assigned the AXI write channels to registers so they could be modified in the FSM. I added 2 new states. one is used to wait for the ALU result and then set all the AXI channels to the correct inputs needed to write to memory. The next state waits for the handshake's valid signal before writing to memory. The FSM will then move onto processing the next instruction.

2. Testing

```
module control_unit_tb();
    parameter WIDTH = 32;
    parameter NUM_REGS = 32;
    bit clk;
    bit rst;
    wire awready;
    bit awvalid;
    bit[19:0] awaddr;
    wire wready;
    bit wvalid;
    bit[31:0] wdata;
    bit bready;
    wire bvalid;
    wire[1:0] bresp;
    wire arready;
    bit arvalid;
    bit[19:0] araddr;
    bit rready;
    wire rvalid;
    wire[31:0] rdata;

    reg _rst;
    reg _awvalid;
    reg[19:0] _awaddr;
    reg _wvalid;
    reg[31:0] _wdata;
    reg _bready;
    reg _arvalid;
    reg[19:0] _araddr;
    reg _rready;

    always #5ns begin
```

```
    clk = ~clk;
end
always @ (posedge clk) begin
    _rst <= rst;
    _awvalid <= awvalid;
    _awaddr <= awaddr;
    _wvalid <= wvalid;
    _wdata <= wdata;
    _bready <= bready;
    _arvalid <= arvalid;
    _araddr <= araddr;
    _rready <= rready;
end
bram_controller my_bram(
    .s_axi_aclk(clk),
    .s_axi_aresetn(~rst),
    .s_axi_araddr(_araddr),
    .s_axi_arprot(0),
    .s_axi_arready(arready),
    .s_axi_arvalid(_arvalid),
    .s_axi_awaddr(_awaddr),
    .s_axi_awprot(0),
    .s_axi_awready(awready),
    .s_axi_awvalid(_awvalid),
    .s_axi_bready(_bready),
    .s_axi_bresp(bresp),
    .s_axi_bvalid(bvalid),
    .s_axi_rdata(rdata),
    .s_axi_rready(_rready),
    .s_axi_rvalid(rvalid),
    .s_axi_wdata(_wdata),
    .s_axi_wready(wready),
    .s_axi_wstrb('b1111),
    .s_axi_wvalid(_wvalid)
);
```

```
reg control_unit_error;
reg [2:0] M_AXI_AWPROT;
control_unit #(.WIDTH(WIDTH), .NUM_REGS(NUM_REGS)) dut (
        .clk(clk),
        .rst(rst),
        .error(control_unit_error),

        .M_AXI_AWADDR(awaddr),
        .M_AXI_AWPROT(awprot),
        .M_AXI_AWVALID(awvalid),
        .M_AXI_AWREADY(awready),

        .M_AXI_WDATA(wdata),
        .M_AXI_WSTRB(wstrb),
        .M_AXI_WVALID(wvalid),
        .M_AXI_WREADY(wready),

        .M_AXI_BRESP(bresp),
        .M_AXI_BVALID(bvalid),
        .M_AXI_BREADY(bready),

        .M_AXI_ARADDR(araddr),
        .M_AXI_ARPROT(arprot),
        .M_AXI_ARVALID(arvalid),
        .M_AXI_ARREADY(arready),

        .M_AXI_RDATA(rdata),
        .M_AXI_RRESP(rresp),
        .M_AXI_RVALID(rvalid),
        .M_AXI_RREADY(rready)
    );
        wire [WIDTH-1:0] regfile [0:NUM_REGS-1];
        genvar i;
    generate
        for (i = 0; i < NUM_REGS; i = i + 1) begin : expose_regs
```

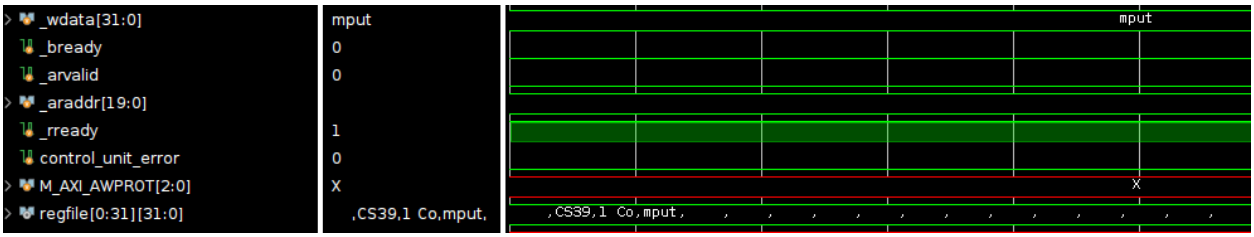
```
            assign regfile[i] = dut.regfile.the_regs[i];
        end
    endgenerate
    reg [7:0] my_memory[511:0];
    initial begin
        rst = 1;
        #20ns;
        rst = 0;
        #20ns;
        $readmemh("/home/ugrad/yc3146/lab3/lab3.srcs/sources_1/new/lab3_bi
nary.hex", my_memory);
        #40ns;
        for (int i = 0; i < 512; i+=4) begin
        awvalid = 1;
        wvalid = 1;
        awaddr = i;
        wdata = {my_memory[i+3], my_memory[i+2], my_memory[i+1], my_memo
ry[i]};
        #20ns;
        awvalid = 0;
        wvalid = 0;
        bready = 1;
        #20ns;
        bready = 0;
        #20ns;
    end

    #20ns;
    // actual test-bench starts here...
    rst = 1;
    #10;
    rst = 0;

    #1000000;
    $finish;
```

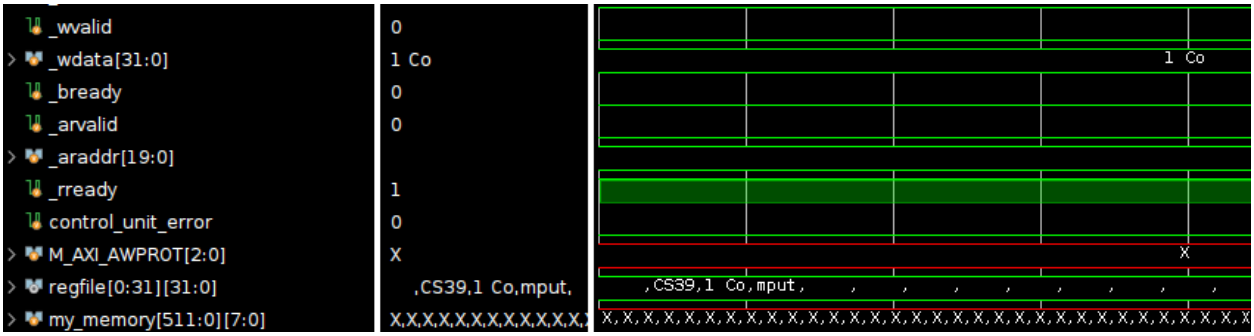
```
end
endmodule
```

I loaded the lab4_binary.hex values into the program. I added breakpoints at where my control unit writes to memory for store word instructions and when the instructions are written to the registers for load word instructions to check that the correct data was being stored back into the registers.

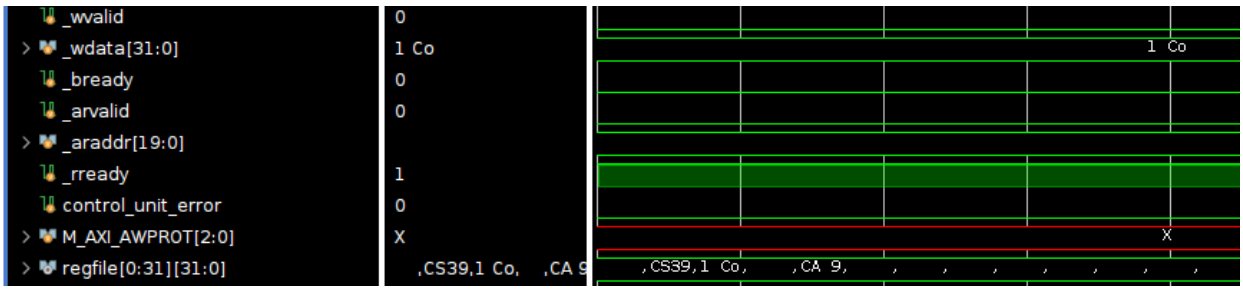
3. Results

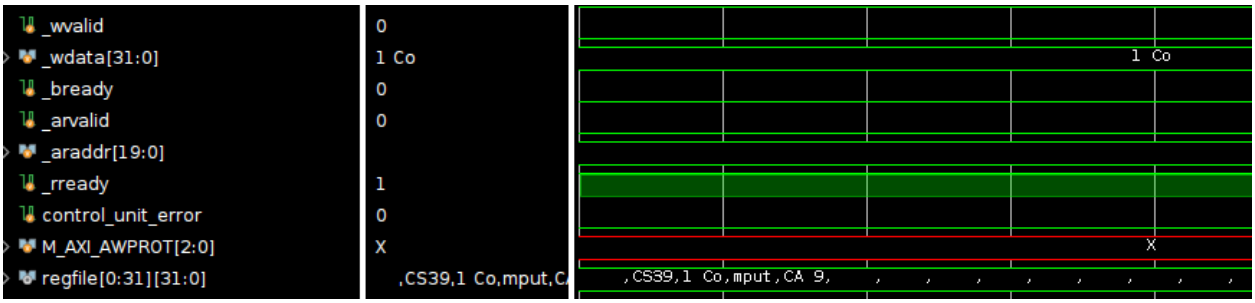
Store Word:





Load Word:





The values `1 Co` and `mput` are correctly written to memory and loaded back in later on.

4. Reflections

The most difficult part of this section was understanding how the AXI handshake works and understanding what each signal should be set to at what time.

Lab 4 Problem 2.3

1. Approach

```
4'd3: begin // STATE 3: EXECUTE
    case (opcode)
        7'b1100011: begin // B-type
            // we use sub as our comparison operator
            alu_control <= 4'b1000;
            alu_op1 <= rs;
            alu_op2 <= rt;

            state <= 5; // jump straight to next instruction state
        end
...

          4'd5: begin // STATE 5: MOVE TO NEXT
              we <= 0;
              if (opcode == 7'b1100011) begin
                  case (funct3)
                      3'h0: begin // BEQ
```

```
                    if (alu_res == 0) begin
                        PC <= PC + {{6{fetched_instruction[31]}}, fetched_instructi
on[31], fetched_instruction[7], fetched_instruction[30:25], fetched_instruction
[11:8], 1'b0};;
                    end else begin
                        PC <= PC + 4;
                    end
                end
                3'h1: begin // BNE
                    if (alu_res != 0) begin
                        PC <= PC + {{7{fetched_instruction[31]}}, fetched_instructi
on[31], fetched_instruction[7], fetched_instruction[30:25], fetched_instruction
[11:8]};;
                    end else begin
                        PC <= PC + 4;
                    end
                end
                3'h4: begin // BLT
                    if ($signed(alu_res) < 0) begin
                        PC <= PC + {{6{fetched_instruction[31]}}, fetched_instructi
on[31], fetched_instruction[7], fetched_instruction[30:25], fetched_instruction
[11:8], 1'b0};;;
                    end else begin
                        PC <= PC + 4;
                    end
                end
                3'h5: begin // BGE
                    if ($signed(alu_res) >= 0) begin
                        PC <= PC + {{6{fetched_instruction[31]}}, fetched_instructi
on[31], fetched_instruction[7], fetched_instruction[30:25], fetched_instruction
[11:8], 1'b0};
                    end else begin
                        PC <= PC + 4;
                    end
                end
                default: PC <= PC + 4;
```

```
                endcase
            end else begin
                PC <= PC + 4;
            end
            state <= 0;
        end

        default: state <= 0;
    endcase
end
```

I added a state in the execute state for B type instructions. This instruction uses the subtraction operation as the comparison operation on rs1 and rs2. This will then immediately jump to state 5 to compute the next instruction to jump to using the immediate value.

2. Testing

```
module control_unit_tb();
    parameter WIDTH = 32;
    parameter NUM_REGS = 32;
    bit clk;
    bit rst;
    wire awready;
    bit awvalid;
    bit[19:0] awaddr;
    wire wready;
    bit wvalid;
    bit[31:0] wdata;
    bit bready;
    wire bvalid;
    wire[1:0] bresp;
    wire arready;
    bit arvalid;
    bit[19:0] araddr;
    bit rready;
```

```
    wire rvalid;
    wire[31:0] rdata;

    reg _rst;
    reg _awvalid;
    reg[19:0] _awaddr;
    reg _wvalid;
    reg[31:0] _wdata;
    reg _bready;
    reg _arvalid;
    reg[19:0] _araddr;
    reg _rready;

    always #5ns begin
        clk = ~clk;
    end
    always @ (posedge clk) begin
        _rst <= rst;
        _awvalid <= awvalid;
        _awaddr <= awaddr;
        _wvalid <= wvalid;
        _wdata <= wdata;
        _bready <= bready;
        _arvalid <= arvalid;
        _araddr <= araddr;
        _rready <= rready;
    end
    bram_controller my_bram(
        .s_axi_aclk(clk),
        .s_axi_aresetn(~rst),
        .s_axi_araddr(_araddr),
        .s_axi_arprot(0),
        .s_axi_arready(arready),
        .s_axi_arvalid(_arvalid),
        .s_axi_awaddr(_awaddr),
        .s_axi_awprot(0),
```

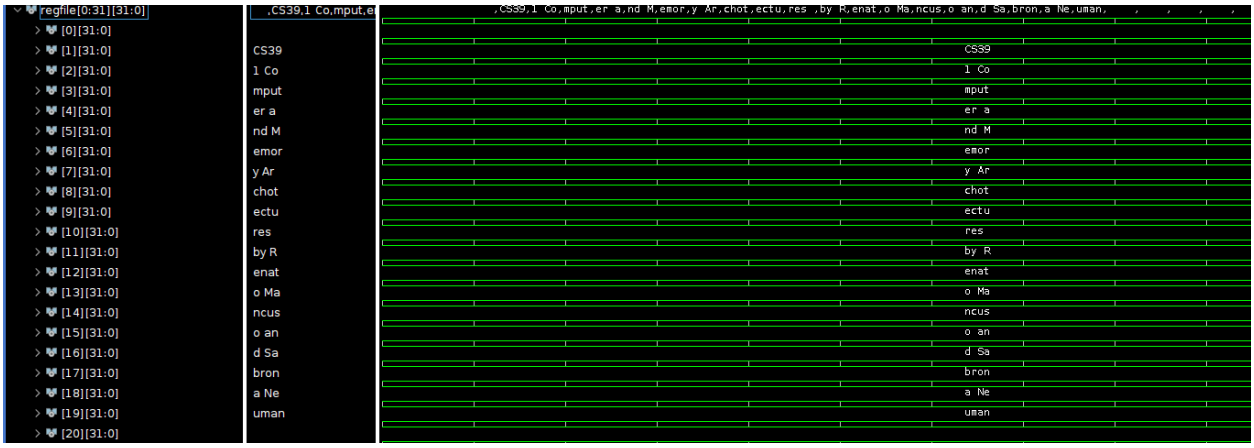
```
    .s_axi_awready(awready),
    .s_axi_awvalid(_awvalid),
    .s_axi_bready(_bready),
    .s_axi_bresp(bresp),
    .s_axi_bvalid(bvalid),
    .s_axi_rdata(rdata),
    .s_axi_rready(_rready),
    .s_axi_rvalid(rvalid),
    .s_axi_wdata(_wdata),
    .s_axi_wready(wready),
    .s_axi_wstrb('b1111),
    .s_axi_wvalid(_wvalid)
);


reg control_unit_error;
reg [2:0] M_AXI_AWPROT;
control_unit #(.WIDTH(WIDTH), .NUM_REGS(NUM_REGS)) dut (
        .clk(clk),
        .rst(rst),
        .error(control_unit_error),

        .M_AXI_AWADDR(awaddr),
        .M_AXI_AWPROT(awprot),
        .M_AXI_AWVALID(awvalid),
        .M_AXI_AWREADY(awready),

        .M_AXI_WDATA(wdata),
        .M_AXI_WSTRB(wstrb),
        .M_AXI_WVALID(wvalid),
        .M_AXI_WREADY(wready),

        .M_AXI_BRESP(bresp),
        .M_AXI_BVALID(bvalid),
        .M_AXI_BREADY(bready),
```

```
        .M_AXI_ARADDR(araddr),
        .M_AXI_ARPROT(arprot),
        .M_AXI_ARVALID(arvalid),
        .M_AXI_ARREADY(arready),

        .M_AXI_RDATA(rdata),
        .M_AXI_RRESP(rresp),
        .M_AXI_RVALID(rvalid),
        .M_AXI_RREADY(rready)
    );
        wire [WIDTH-1:0] regfile [0:NUM_REGS-1];
        genvar i;
    generate
        for (i = 0; i < NUM_REGS; i = i + 1) begin : expose_regs
            assign regfile[i] = dut.regfile.the_regs[i];
        end
    endgenerate
  reg [7:0] my_memory[511:0];
  initial begin
    rst = 1;
    #20ns;
    rst = 0;
    #20ns;
    $readmemh("/home/ugrad/yc3146/lab3/lab3.srcs/sources_1/new/lab3_bi
nary.hex", my_memory);
    #40ns;
    for (int i = 0; i < 512; i+=4) begin
    awvalid = 1;
    wvalid = 1;
    awaddr = i;
    wdata = {my_memory[i+3], my_memory[i+2], my_memory[i+1], my_memo
ry[i]};
    #20ns;
    awvalid = 0;
    wvalid = 0;
    bready = 1;
```

```
        #20ns;
        bready = 0;
        #20ns;
    end

    #20ns;
    // actual test-bench starts here...
    rst = 1;
    #10;
    rst = 0;

    #1000000;
    $finish;

end
endmodule
```

Since this is the final problem of the lab, I tested my control unit against the lab4_binary.hex program.

3. Results



Every register is correctly assigned the value in the string “CS391 Computer and Memory Archotectures by Renato Mancuso and Sabrona Neuman”

4. Reflections

Creating the states and the logic for the branch instructions was straightforward. I had some difficulty adding the correct value to the PC when a branch was taken because a 0 bit was needed at the end of the immediate as an offset.